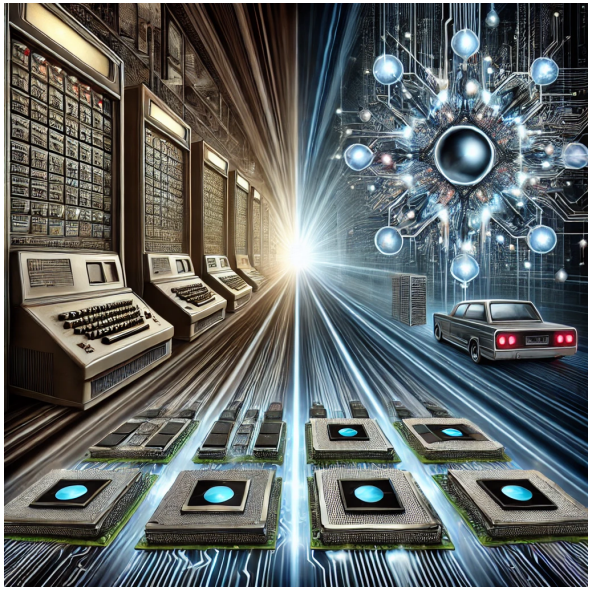
Language Models for Law and Social Science

9. LLM Alignment

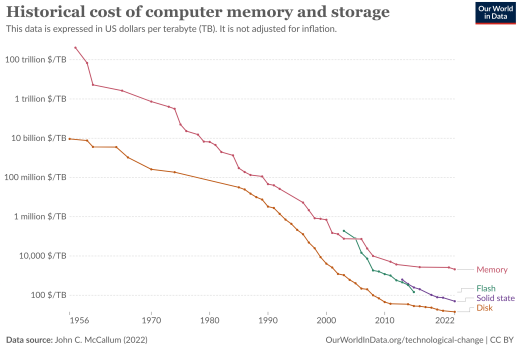


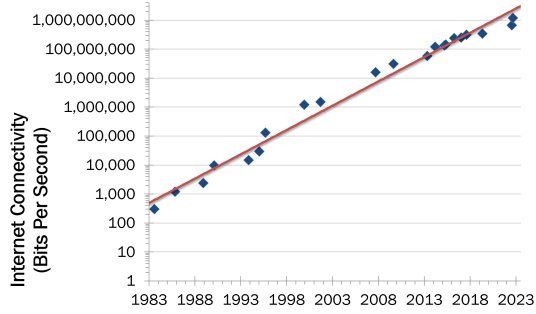
How did we get to LLMs and AI?
A brief history

Data: Storage and Bandwidth

Data: Storage and Bandwidth

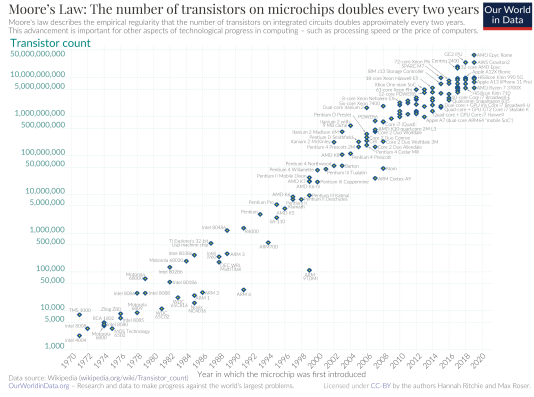
- Key data technologies from the 1900s:
  - hard disks for efficient storage
  - relational database systems for efficient access and linking
  - the internet – for sharing, and then for collection
  - digitization of analog data – especially optical character recognition (OCR)





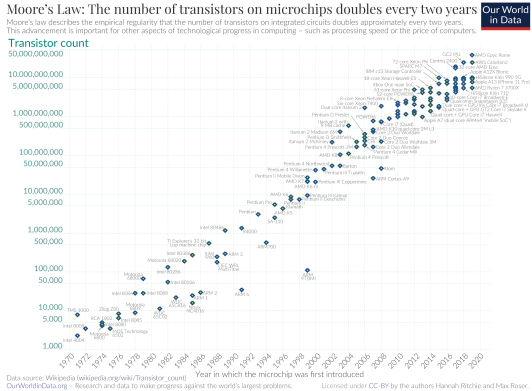
Compute: Processors and ML Models

- Developments in procesors and RAM:
  - single-core CPU (1970s-1990s)
  - multi-core CPU (2000s)
  - GPUs (2010s-2020s)



Compute: Processors and ML Models

- Developments in procesors and RAM:
  - single-core CPU (1970s-1990s)
  - multi-core CPU (2000s)
  - GPUs (2010s-2020s)



- Developments in ML models:
  - classical ML, including regression, decision trees, SVMs (1960s-1980s)
  - CPU-based deep learning, including CNNs for computer vision and word embeddings (2000s-2010s)
  - GPU-based deep learning $\rightarrow$ emergence of transformers (late 2010s)

Data + ML = Better Digitization

- Early OCR quality was very poor except with high-quality scans.
- Early solution: better and cheaper scanners (e.g. Google Books).



Data + ML = Better Digitization

- Early OCR quality was very poor except with high-quality scans.
- Early solution: better and cheaper scanners (e.g. Google Books).



- AI-based digitization with pre-trained vision/language models can fill in gaps based on knowledge and context.
  - provides high-quality OCR even with handwriting, or with damaged historical documents.
  - can extract structured data, e.g. tables and charts.

Data + ML = Better Digitization

- Early OCR quality was very poor except with high-quality scans.
- Early solution: better and cheaper scanners (e.g. Google Books).



- AI-based digitization with pre-trained vision/language models can fill in gaps based on knowledge and context.
  - provides high-quality OCR even with handwriting, or with damaged historical documents.
  - can extract structured data, e.g. tables and charts.
- Meanwhile:
  - multilingual AI's deliver near-flawless machine translation.
  - AI-based audio transcription opens up countless hours of voice recordings for digitization.

Big data & big compute are preconditions for AI

- With GPT-1 (2018), GPT-2 (2019), and GPT-3 (2020), OpenAI researchers gradually added more "neurons" so the LLMs can learn more about language.
- Further, the models are pre-trained (that is, they have read) a much larger and more diverse corpus:
  - GPT-2: 8 billion words
  - GPT-3: 114 billion words

Big data & big compute are preconditions for AI

- With GPT-1 (2018), GPT-2 (2019), and GPT-3 (2020), OpenAI researchers gradually added more "neurons" so the LLMs can learn more about language.
- Further, the models are pre-trained (that is, they have read) a much larger and more diverse corpus:
  - GPT-2: 8 billion words
  - GPT-3: 114 billion words
  - **2023-era LLMs (e.g. ChatGPT): 10 trillion words!**

But are big data and compute sufficient for AI?

But are big data and compute sufficient for AI?

Data (the whole internet & millions of digitized books)

+

Compute (billions of parameters)

+

ML (transformer architecture)

=

But are big data and compute sufficient for AI?

Data (the whole internet & millions of digitized books)

+

Compute (billions of parameters)

+

ML (transformer architecture)

=

AI systems that are getting close to human-level intelligence (e.g. GPT-3)

The Final Ingredient

Data (the whole internet & millions of digitized books)

+

Compute (billions of parameters)

+

ML (transformer architecture)

+

The Final Ingredient

Data (the whole internet & millions of digitized books)

+

Compute (billions of parameters)

+

ML (transformer architecture)

+

?

How do children learn language?



How do children learn language?

1. They learn to repeat what they hear.

How do children learn language?

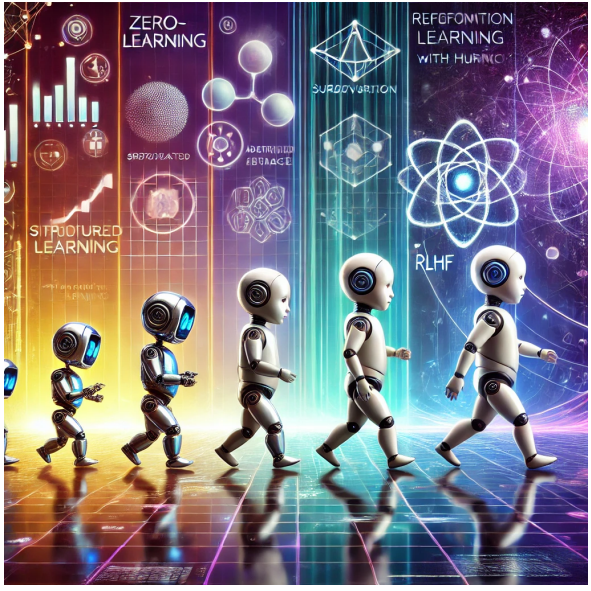
1. They learn to repeat what they hear.
2. They learn some explicit rules from parents/teachers.

How do children learn language?

1. They learn to repeat what they hear.
2. They learn some explicit rules from parents/teachers.
3. They get positive and negative feedback when they say something right or wrong.

How do ~~children~~ **aligned LLMs** learn language?

1. They learn to repeat what they hear.
2. They learn some explicit rules from parents/teachers.
3. They get positive and negative feedback when they say something right or wrong.



From zero-shot learning to human preference alignment

Outline

From GPT to ChatGPT

Emergent zero-shot learning (GPT-1, GPT-2)

- As far back as GPT (2018) but especially with GPT-2 ("Language models are unsupervised multitask learners", 2019), researchers observed emergent zero-shot learning.
- e.g.: Passage: [wikipedia excerpt for X]
  - Q: Where was X born? $\rightarrow$A: ...
  - TLDR: ...

Main source for these slides: Jesse Mu's CS224n slides

`https://cs.stanford.edu/~muj/`;
`https://web.stanford.edu/class/cs224n/slides/cs224n-2023-lecture11-prompting-rlhf.pdf`

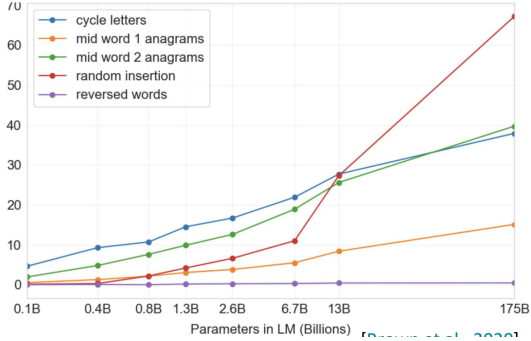
Emergent few-shot learning (GPT-3)

"Few-shot learning": Specify a task and give examples before the example.

- became especially powerful with GPT-3 ("Language models are few-shot learners", 2020)

```
Translate English to French:
sea otter => loutre de mer
peppermint => menthe poivrée
plush girafe => girafe peluche
cheese =>
```

Synthetic "word unscrambling" tasks, 100-shot



Chain-of-thought prompting

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ✗

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

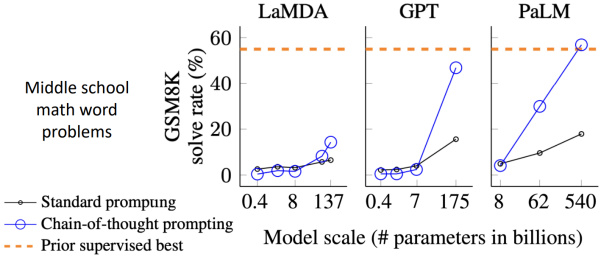
A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. ✓

Chain-of-thought prompting is an emergent property of model scale



Language modeling ≠ assisting users

PROMPT	*Explain the moon landing to a 6 year old in a few sentences.*
COMPLETION	GPT-3

```
Explain the theory of gravity to a 6 year old.

Explain the theory of relativity to a 6 year old in a few sentences.

Explain the big bang theory to a 6 year old.

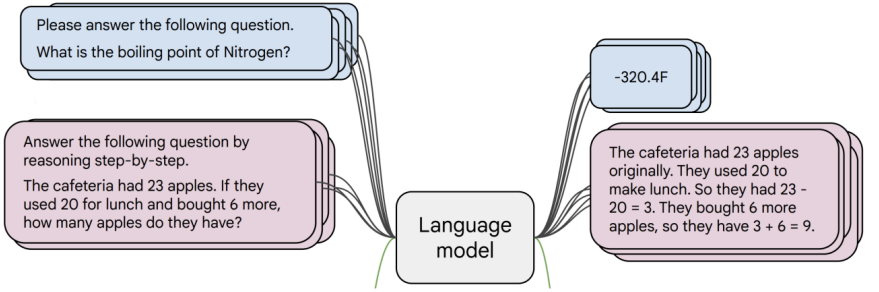
Explain evolution to a 6 year old.
```

Language models are not *aligned* with user intent [Ouyang et al., 2022].

Instruction fine-tuning (e.g. FLAN-T5)

Instruction finetuning

- **Collect examples** of (instruction, output) pairs across many tasks and finetune an LM



- Evaluate on **unseen tasks**



Instruction fine-tuning

Model input (Disambiguation QA)

Q: In the following sentences, explain the antecedent of the pronoun (which thing the pronoun refers to), or state that it is ambiguous.

Sentence: The reporter and the chef will discuss their favorite dishes.

Options:
(A) They will discuss the reporter's favorite dishes
(B) They will discuss the chef's favorite dishes
(C) Ambiguous

A: Let's think step by step.

Before instruction finetuning

The reporter and the chef will discuss their favorite dishes.
The reporter and the chef will discuss the reporter's favorite dishes.
The reporter and the chef will discuss the chef's favorite dishes.
The reporter and the chef will discuss the reporter's and the chef's favorite dishes.

✖ **(doesn't answer question)**

After instruction finetuning

The reporter and the chef will discuss their favorite dishes does not indicate whose favorite dishes they will discuss. So, the answer is (C). ✔

Limitations of instruction fine-tuning

1. Collecting task data is expensive.
2. Some important tasks cannot be written as instructions with correct answers. e.g., story generation.
3. Language models penalize all mistakes equally at the token level; but some mistakes are more important than others.
4. Human written answers might be suboptimal.

Aligning language models with reinforcement learning

- Say we have a language model $p_\theta(y|x)$ with learnable parameters θ, giving the probability of output y given input/task x, with samples represented by $\hat{y}(x)$
- Further, for each LM task, we have a reward function $U(y, x) \in \mathbb{R}$.
  - e.g., human scoring/evaluation of LM outputs
  - will come back to define $U(\cdot)$, but importantly, does not have to be differentiable.

Aligning language models with reinforcement learning

- Say we have a language model $p_\theta(y|x)$ with learnable parameters θ, giving the probability of output y given input/task x, with samples represented by $\hat{y}(x)$
- Further, for each LM task, we have a reward function $U(y, x) \in \mathbb{R}$.
  - e.g., human scoring/evaluation of LM outputs
  - will come back to define $U(\cdot)$, but importantly, does not have to be differentiable.
- We want to choose θ maximize the expected reward from generated samples $\hat{y}_\theta(x) \sim p_\theta(y|x)$:

$$\theta^* = \arg\max_\theta \mathbb{E}\{U(\hat{y}_\theta(x), x)\}$$

Aligning language models with reinforcement learning

- Say we have a language model $p_\theta(y|x)$ with learnable parameters θ, giving the probability of output y given input/task x, with samples represented by $\hat{y}(x)$
- Further, for each LM task, we have a reward function $U(y, x) \in \mathbb{R}$.
  - e.g., human scoring/evaluation of LM outputs
  - will come back to define $U(\cdot)$, but importantly, does not have to be differentiable.
- We want to choose θ maximize the expected reward from generated samples $\hat{y}_\theta(x) \sim p_\theta(y|x)$:

$$\theta^* = \arg\max_\theta \mathbb{E}\{U(\hat{y}_\theta(x), x)\}$$

- θ^* can be learned via gradient ascent (follow the reward gradient):

$$\theta_{t+1} := \theta_t + \alpha \nabla_{\theta_t} \mathbb{E}\{U(\hat{y}_\theta(x), x)\}$$

  - $\rightarrow$ solvable by policy gradient methods from reinforcement learning.

RL in 2 slides (1)

- For a sample of generations S we have

$$\nabla_\theta \mathbb{E}\{U(\hat{y}_\theta(x), x)\} = \nabla_\theta \sum_{y \in S} U(\hat{y}_\theta(x), x) p_\theta(y|x) = \sum_{y \in S} U(\hat{y}_\theta(x), x) \nabla_\theta p_\theta(y|x)$$

RL in 2 slides (1)

- For a sample of generations S we have

$$\nabla_\theta \mathbb{E}\{U(\hat{y}_\theta(x), x)\} = \nabla_\theta \sum_{y \in S} U(\hat{y}_\theta(x), x) p_\theta(y|x) = \sum_{y \in S} U(\hat{y}_\theta(x), x) \nabla_\theta p_\theta(y|x)$$

- "log derivative trick":

$$\nabla_\theta \log p_\theta(y|x) = \frac{1}{p_\theta(y|x)} \nabla_\theta p_\theta(y|x) \rightarrow \nabla_\theta p_\theta(y|x) = p_\theta(y|x) \nabla_\theta \log p_\theta(y|x)$$

RL in 2 slides (1)

- For a sample of generations S we have

$$\nabla_\theta \mathbb{E}\{U(\hat{y}_\theta(x), x)\} = \nabla_\theta \sum_{y \in S} U(\hat{y}_\theta(x), x) p_\theta(y|x) = \sum_{y \in S} U(\hat{y}_\theta(x), x) \nabla_\theta p_\theta(y|x)$$

- "log derivative trick":

$$\nabla_\theta \log p_\theta(y|x) = \frac{1}{p_\theta(y|x)} \nabla_\theta p_\theta(y|x) \rightarrow \nabla_\theta p_\theta(y|x) = p_\theta(y|x) \nabla_\theta \log p_\theta(y|x)$$

- Then:

$$\begin{aligned} \sum_{y \in S} U(\hat{y}_\theta(x), x) \nabla_\theta p_\theta(y|x) &= \sum_{y \in S} p_\theta(y|x) U(\hat{y}_\theta(x), x) \nabla_\theta \log p_\theta(y|x) \\ &= \mathbb{E}\{U(\hat{y}_\theta(x), x)\} \nabla_\theta \log p_\theta(y|x) \end{aligned}$$

- i.e.,

$$\nabla_\theta \mathbb{E}\{U(\hat{y}_\theta(x), x)\} = \mathbb{E}\{U(\hat{y}_\theta(x), x)\} \nabla_\theta \log p_\theta(y|x)$$

RL in 2 slides (2)

- From previous slide

$$\nabla_\theta \mathbb{E}\{U(\hat{y}_\theta(x), x)\} = \mathbb{E}\{U(\hat{y}_\theta(x), x)\} \nabla_\theta \log p_\theta(y|x)$$

- LHS cannot be estimated because $U(\cdot)$ is not differentiable.

RL in 2 slides (2)

- From previous slide

$$\nabla_\theta \mathbb{E}\{U(\hat{y}_\theta(x), x)\} = \mathbb{E}\{U(\hat{y}_\theta(x), x)\} \nabla_\theta \log p_\theta(y|x)$$

- LHS cannot be estimated because $U(\cdot)$ is not differentiable.
- RHS can be estimated by Monte Carlo:

$$\approx \frac{1}{m} \sum_{i=1}^{m} U(y_i, x) \nabla_\theta \log p_\theta(y_i|x)$$

where the y_i are sampled from the LM.

RL in 2 slides (2)

- From previous slide

$$\nabla_\theta \mathbb{E}\{U(\hat{y}_\theta(x), x)\} = \mathbb{E}\{U(\hat{y}_\theta(x), x)\} \nabla_\theta \log p_\theta(y|x)$$

- LHS cannot be estimated because $U(\cdot)$ is not differentiable.
- RHS can be estimated by Monte Carlo:

$$\approx \frac{1}{m} \sum_{i=1}^{m} U(y_i, x) \nabla_\theta \log p_\theta(y_i|x)$$

  where the y_i are sampled from the LM.
- i.e., the gradient ascent update is

$$\theta_{t+1} := \theta_t + \alpha \frac{1}{m} \sum_{i=1}^{m} U(y_i, x) \nabla_\theta \log p_\theta(y_i|x)$$

  - if $U(y_i, x)$ is positive, change θ to increase $p_\theta(y_i|x)$
  - if $U(y_i, x)$ is negative, change θ to decrease $p_\theta(y_i|x)$
- will converge to optimum θ^* (under some assumptions)

How to model human preferences $U(\cdot)$

Problem 1: human-in-the-loop is too expensive to measure $U(y, x)$ for all y, x.

- solution: train a text regression model $U_\phi(y, x)$ to predict U from y, x, use that instead of ground truth.
  - E.g., for Stiennon et al (2020), fine-tune a GPT model and add a linear regression output layer to predict likert scale values (1-7), output $\hat{U}_\phi(y, x)$

How to model human preferences $U(\cdot)$

Problem 1: human-in-the-loop is too expensive to measure $U(y, x)$ for all y, x.

- solution: train a text regression model $U_\phi(y, x)$ to predict U from y, x, use that instead of ground truth.
  - E.g., for Stiennon et al (2020), fine-tune a GPT model and add a linear regression output layer to predict likert scale values (1-7), output $\hat{U}_\phi(y, x)$

Problem 2: human preference scores are noisy and not well calibrated:

- solution: use pairwise comparisons (**revealed preference!**) rather than individual scores.

How to model human preferences $U(\cdot)$

Problem 1: human-in-the-loop is too expensive to measure $U(y,x)$ for all y,x.

- solution: train a text regression model $U_\phi(y,x)$ to predict U from y,x, use that instead of ground truth.
  - E.g., for Stiennon et al (2020), fine-tune a GPT model and add a linear regression output layer to predict likert scale values (1-7), output $\hat{U}_\phi(y,x)$

Problem 2: human preference scores are noisy and not well calibrated:

- solution: use pairwise comparisons (**revealed preference!**) rather than individual scores.
  - Learn scores $\hat{U}_\phi(y,x)$ to predict which of two LM outputs y_0, y_1 is preferred.
  - e.g., with a Bradley-Terry model: minimize the paired comparison loss

$$L(\phi) = \frac{1}{|D|} \sum_D \text{sigmoid}(U_\phi(y_0,x) - U_\phi(y_1,x))$$

where D is the dataset of pairs (y_0, y_1), with y_0 giving the preferred LM output.

How to model human preferences $U(\cdot)$

Problem 1: human-in-the-loop is too expensive to measure $U(y,x)$ for all y,x.

- solution: train a text regression model $U_\phi(y,x)$ to predict U from y,x, use that instead of ground truth.
  - E.g., for Stiennon et al (2020), fine-tune a GPT model and add a linear regression output layer to predict likert scale values (1-7), output $\hat{U}_\phi(y,x)$

Problem 2: human preference scores are noisy and not well calibrated:

- solution: use pairwise comparisons (**revealed preference!**) rather than individual scores.
  - Learn scores $\hat{U}_\phi(y,x)$ to predict which of two LM outputs y_0, y_1 is preferred.
  - e.g., with a Bradley-Terry model: minimize the paired comparison loss

$$L(\phi) = \frac{1}{|D|} \sum_D \text{sigmoid}(U_\phi(y_0,x) - U_\phi(y_1,x))$$

where D is the dataset of pairs (y_0,y_1), with y_0 giving the preferred LM output.

- $\hat{U}_\phi(y,x)$ is a fine-tuned LLM outputting predicted probability that s is preferred in average pairwise match-up.
  - **It is the implicit user utility function that ChatGPT is trying to maximize every time it replies.**

Instruct GPT: Data Collection

- Annotation team:
  - 40 annotators from Upwork and ScaleAI.
  - Annotators had to pass test annotation task on flagging sensitive content, completion quality, demo writing, and diversity of group membership.

Table 12: Labeler demographic data

What gender do you identify as?	
Male	50.0%
Female	44.4%
Nonbinary / other	5.6%
What ethnicities do you identify as?	
White / Caucasian	31.6%
Southeast Asian	52.6%
Indigenous / Native American / Alaskan Native	0.0%
East Asian	5.3%
Middle Eastern	0.0%
Latinx	15.8%
Black / of African descent	10.5%
What is your nationality?	
Filipino	22%
Bangladeshi	22%
American	17%
Albanian	5%
Brazilian	5%
Canadian	5%
Colombian	5%
Indian	5%
Uruguayan	5%
Zimbabwean	5%

What is your age?	
18-24	26.3%
25-34	47.4%
35-44	10.5%
45-54	10.5%
55-64	5.3%
65+	0%
What is your highest attained level of education?	
Less than high school degree	0%
High school degree	10.5%
Undergraduate degree	52.6%
Master's degree	36.8%
Doctorate degree	0%

Submit Skip « Page 3 / 11 » Total time: 05:39

Instruction

Include output

Summarize the following news article:

====
{article}
====

Output A

summary1

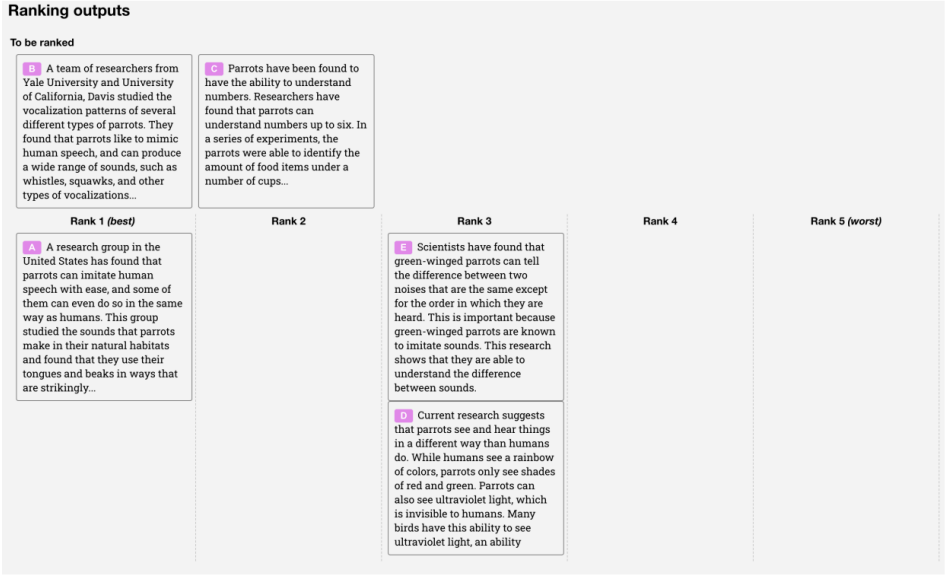
Rating (1 = worst, 7 = best)

1 2 3 4 5 6 7

Fails to follow the correct instruction / task **?**	Yes	No
Inappropriate for customer assistant **?**	Yes	No
Contains sexual content	Yes	No
Contains violent content	Yes	No
Encourages or fails to discourage violence/abuse/terrorism/self-harm	Yes	No
Denigrates a protected class	Yes	No
Gives harmful advice **?**	Yes	No
Expresses moral judgment	Yes	No

Notes

(Optional) notes



Nathan Lambert: "From the Llama 2 numbers and other rumors, it's likely Meta spent \$10-20 million plus on preference data. This is just for the final model and may not include extensive experimentation and evaluation."

RLHF: Reinforcement Learning with Human Feedback

- Ingredients:
  - A pre-trained language model $p_\theta^{PT}(y|x)$, with parameters θ now frozen
  - a reward model $U_\phi(y, x)$

RLHF: Reinforcement Learning with Human Feedback

- Ingredients:
  - A pre-trained language model $p_\theta^{PT}(y|x)$, with parameters θ now frozen
  - a reward model $U_\phi(y,x)$
- RLHF:
  - make a copy $p_\psi^{RL} = p_\theta^{PT}(y|x)$, with newly trainable parameters ψ
  - Use RL with samples $S \sim p_\psi^{RL}$ to learn new parameters ψ, to solve

$$\psi^* = \arg\max_\psi \sum_{y \in S} \left(U_\phi(y,x) - \beta \log\left(\frac{p_\psi^{RL}(y|x)}{p^{PT}(y|x)}\right) \right)$$

  - second term penalizes diverging from pre-trained model $p^{PT}(y|x)$.
  - calibrated by hyperparameter β (e.g. $\beta = 0.05$ in Stiennen et al 2020).

Instruct GPT (Ouyang et al 2022)

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



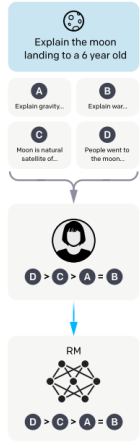
Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

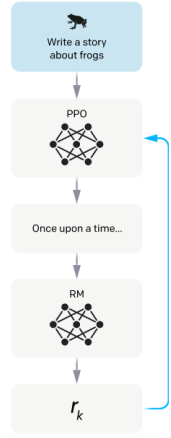
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

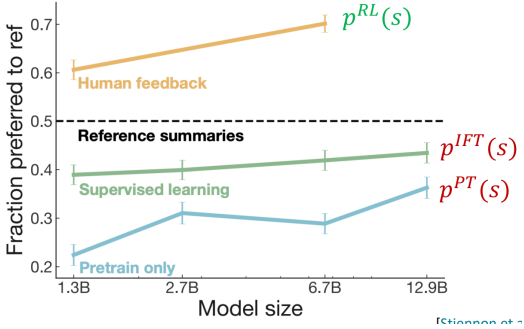
The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



Summarization with RLHF (Stiennon et al 2020)



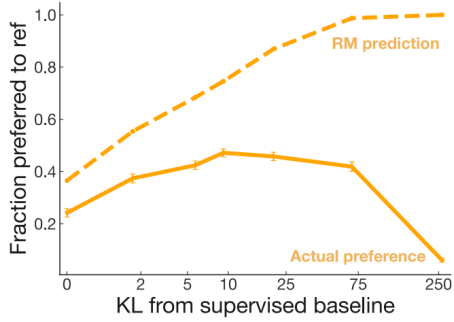


Figure 5: Preference scores versus degree of reward model optimization. Optimizing against the reward model initially improves summaries, but eventually overfits, giving worse summaries.

SFT+RLHF > SFT

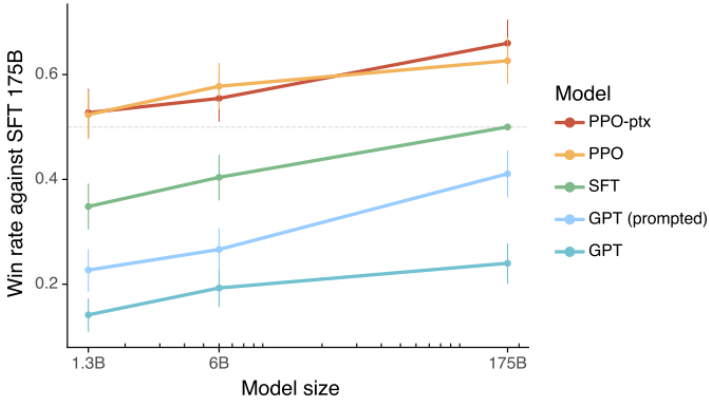


Figure 1: Human evaluations of various models on our API prompt distribution, evaluated by how often outputs from each model were preferred to those from the 175B SFT model. Our InstructGPT models (PPO-ptx) as well as its variant trained without pretraining mix (PPO) significantly outperform the GPT-3 baselines (GPT, GPT prompted); outputs from our 1.3B PPO-ptx model are preferred to those from the 175B GPT-3. Error bars throughout the paper are 95% confidence intervals.



Figure 11: Evolution of Llama 2-Chat. We show the evolution after multiple iterations fine-tuning for the win-rate % of Llama 2-Chat compared to ChatGPT. *Left*: the judge is our reward model, which may favor our model, and *right*, the judge is GPT-4, which should be more neutral.

Why RLHF? (Goldberg 2023)

`https://gist.github.com/yoavg/6bff0fecd65950898eba1bb321cfbd81`

- Why is RLHF needed instead of autoregressive pre-training (APT) or instruction fine-tuning (IFT)?
- Yoav Goldberg discusses three interesting reasons:
  - Flexibility of wording
  - Positive *and* negative feedback
  - Preventing hallucination

Why RLHF 1: Flexibility of Wording

- With autoregressive pre-training, or instruction fine-tuning (without RL), LM has to replicate the exact wording.
- But with human language, there are many valid ways to convey the same correct answer.
- APT and IFT "punish" the model for slight deviations from the prescribed text. The human-provided examples might insist on phrasing which is hard for the model to learn, while the model already knows how to produce an alternative---and equally valid---answer.

Why RLHF 2: Negative Feedback

- SFT models only allow positive feedback; RLHF allows negative feedback.
  - in formal learning theory, negative feedback is powerful – without it, an adversarial teacher can mislead the learner by withhodling some important examples.
- GPT-3 knows what to say; Chat-GPT knows what **not** to say.
  - this makes LM training much closer to how humans learn (language).

Why RLHF 3: Preventing Hallucination

- LMs have 3 main modes of interaction:
  - (1) text-grounded: answer a question or manipulate a provided document
  - (2) knowledge-seeking: answer a question from the LM's knowledge base
  - (3) creative: generate something new

Why RLHF 3: Preventing Hallucination

- LMs have 3 main modes of interaction:
  - (1) text-grounded: answer a question or manipulate a provided document
  - (2) knowledge-seeking: answer a question from the LM's knowledge base
  - (3) creative: generate something new
- For (2) knowledge-seeking, it is crucial that the model can refuse to answer when it doesn't know.
  - SFT teaches the model to lie – if it doesn't know, it will make something up.
  - RLHF can prevent this.

Open Questions on RLHF

- How reliable are (models of) human preferences?
- RLHF can produce chatbots that "seem" right but aren't.
- How to treat abstaining ("I don't know" answers)? Should these be penalized differently?
- Human alignment labeling is cheaper than training GPT-3, but still quite expensive.
  - Fine-tuning new models on ChatGPT outputs will not produce a ChatGPT clone.
  - Interesting alternatives: crowd funding, AI feedback

DPO: Direct Preference Optimization

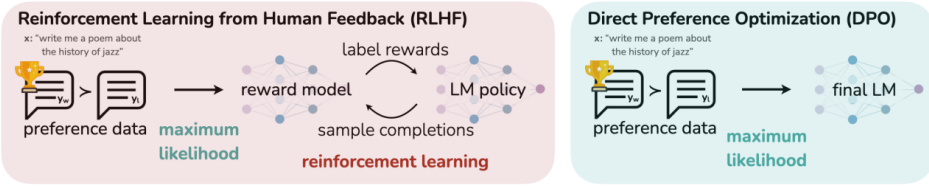


Figure 1: **DPO optimizes for human preferences while avoiding reinforcement learning.** Existing methods for fine-tuning language models with human feedback first fit a reward model to a dataset of prompts and human preferences over pairs of responses, and then use RL to find a policy that maximizes the learned reward. In contrast, DPO directly optimizes for the policy best satisfying the preferences with a simple classification objective, fitting an *implicit* reward model whose corresponding optimal policy can be extracted in closed form.

DPO: Direct Preference Optimization

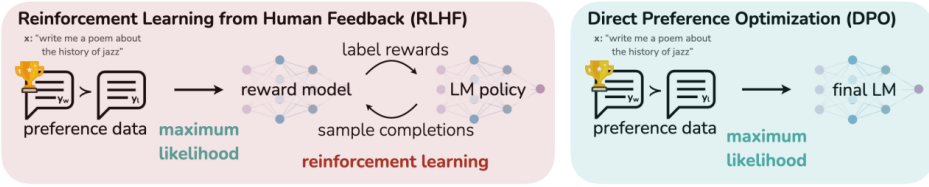


Figure 1: **DPO optimizes for human preferences while avoiding reinforcement learning.** Existing methods for fine-tuning language models with human feedback first fit a reward model to a dataset of prompts and human preferences over pairs of responses, and then use RL to find a policy that maximizes the learned reward. In contrast, DPO directly optimizes for the policy best satisfying the preferences with a simple classification objective, fitting an *implicit* reward model whose corresponding optimal policy can be extracted in closed form.

What does the DPO update do? For a mechanistic understanding of DPO, it is useful to analyze the gradient of the loss function $\mathcal{L}_{\text{DPO}}$. The gradient with respect to the parameters θ can be written as:

$$\nabla_\theta \mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) =$$
$$-\beta \mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\Bigg[\underbrace{\sigma(\hat{r}_\theta(x,y_l) - \hat{r}_\theta(x,y_w))}_{\text{higher weight when reward estimate is wrong}}\Bigg[\underbrace{\nabla_\theta \log \pi(y_w \mid x)}_{\text{increase likelihood of } y_w} - \underbrace{\nabla_\theta \log \pi(y_l \mid x)}_{\text{decrease likelihood of } y_l}\Bigg]\Bigg],$$

where $\hat{r}_\theta(x,y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ is the reward implicitly defined by the language model π_θ and reference model π_{ref}

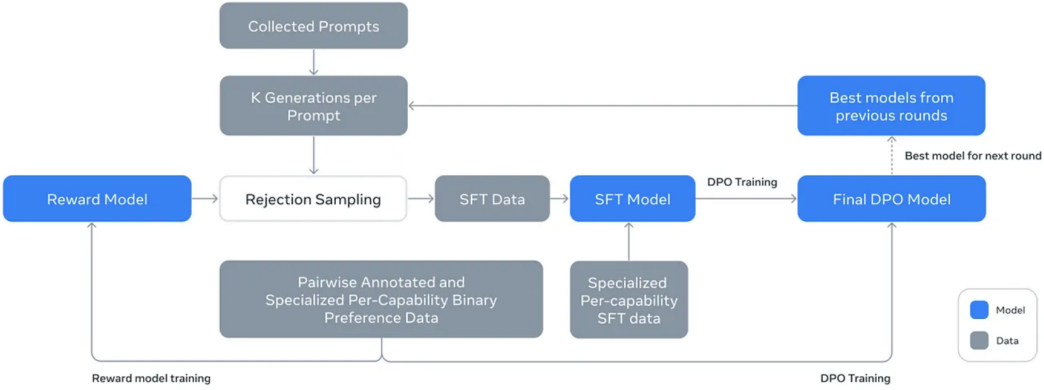
- LlaMa 3 uses DPO and then PPO.

ChatGPT = InstructGPT + Conversations

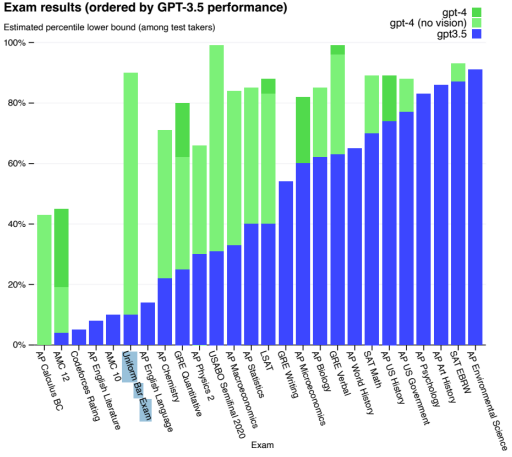
We trained this model using Reinforcement Learning from Human Feedback (RLHF), using the same methods as InstructGPT, but with slight differences in the data collection setup. We trained an initial model using supervised fine-tuning: human AI trainers provided conversations in which they played both sides—the user and an AI assistant. We gave the trainers access to model-written suggestions to help them compose their responses. We mixed this new dialogue dataset with the InstructGPT dataset, which we transformed into a dialogue format.

To create a reward model for reinforcement learning, we needed to collect comparison data, which consisted of two or more model responses ranked by quality. To collect this data, we took conversations that AI trainers had with the chatbot. We randomly selected a model-written message, sampled several alternative completions, and had AI trainers rank them. Using these reward models, we can fine-tune the model using Proximal Policy Optimization. We performed several iterations of this process.

LlaMa 3 Alignment Pipeline



GPT-4 = ChatGPT + 10T tokens + 1.7T params (MoE)



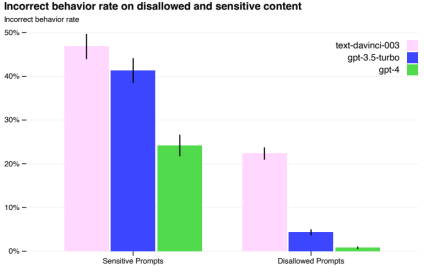


Figure 9. Rate of incorrect behavior on sensitive and disallowed prompts. Lower values are better. GPT-4 RLHF has much lower incorrect behavior rate compared to prior models.

The Final Ingredient

Data (the whole internet & millions of digitized books)

+

Compute (billions of parameters)

+

ML (transformer architecture)

+

The Final Ingredient

Data (the whole internet & millions of digitized books)

+

Compute (billions of parameters)

+

ML (transformer architecture)

+

Human Preference Alignment (Instruction Tuning and Learning from Feedback)

=

AI systems that often surpass human-level intelligence
(GPT-4+, Gemini, Claude, etc.)

Sparks of Artificial General Intelligence: Early experiments with GPT-4

Sébastien Bubeck Varun Chandrasekaran Ronen Eldan Johannes Gehrke
Eric Horvitz Ece Kamar Peter Lee Yin Tat Lee Yuanzhi Li Scott Lundberg
Harsha Nori Hamid Palangi Marco Tulio Ribeiro Yi Zhang

Microsoft Research

Abstract

Artificial intelligence (AI) researchers have been developing and refining large language models (LLMs) that exhibit remarkable capabilities across a variety of domains and tasks, challenging our understanding of learning and cognition. The latest model developed by OpenAI, GPT-4 [Ope23], was trained using an unprecedented scale of compute and data. In this paper, we report on our investigation of an early version of GPT-4, when it was still in active development by OpenAI. We contend that (this early version of) GPT-4 is part of a new cohort of LLMs (along with ChatGPT and Google's PaLM for example) that exhibit more general intelligence than previous AI models. We discuss the rising capabilities and implications of these models. We demonstrate that, beyond its mastery of language, GPT-4 can solve novel and difficult tasks that span mathematics, coding, vision, medicine, law, psychology and more, without needing any special prompting. Moreover, in all of these tasks, GPT-4's performance is strikingly close to human-level performance, and often vastly surpasses prior models such as ChatGPT. Given the breadth and depth of GPT-4's capabilities, we believe that it could reasonably be viewed as an early (yet still incomplete) version of an artificial general intelligence (AGI) system. In our exploration of GPT-4, we put special emphasis on discovering its limitations, and we discuss the challenges ahead for advancing towards deeper and more comprehensive versions of AGI, including the possible need for pursuing a new paradigm that moves beyond next-word prediction. We conclude with reflections on societal influences of the recent technological leap and future research directions.

Prompt: Can you write a proof that there are infinitely many primes, with every line that rhymes?

ChatGPT:

Sure! Here is a proof that there are infinitely many primes,
with each line of the proof rhyming:

Consider the set of all primes.
Call this set P and let it start with just two, the first prime.
If we take all the primes in P and add one,
We get a new set of numbers that has never been done.

This set, call it S, is composed of only composite numbers.
No primes will be found among them, that's for sure.
But we know that there must be at least one prime,
Since S is infinite, and that much is clear.

If we take the smallest number in S that is not in P,
And call it p, we can add it to our set, don't you see?
But this process can be repeated indefinitely.
Thus, our set P must also be infinite, you'll agree.

So there you have it, a proof that's quite solid.
There are infinitely many primes, that much we can see.
No need to be coy, no need to be contrite.
Infinity is a big number, just like P.

GPT-4:

Yes, I think I can,
Though it might take a clever plan.
I'll start by noting Euclid's proof,
Which shows that primes aren't just aloof.
Assume we have a finite list,
Of primes, and that none have been missed.
Multiply them all together,
And add one, just to be clever.
The result will be a number,
That has no prime factors, oh wonder!
But every number has a prime,
That divides it, it's just a matter of time.
So we've found a contradiction,
And our finite list needs eviction.
There must be infinitely many primes,
And that's the end of my rhyming lines.s

Prompt: Can you write a proof that there are infinitely many primes, with every line that rhymes?

ChatGPT:

```
Sure! Here is a proof that there are infinitely many primes,
with each line of the proof rhyming:

Consider the set of all primes.
Call this set P and let it start with just two, the first
prime.
If we take all the primes in P and add one,
We get a new set of numbers that has never been done.

This set, call it S, is composed of only composite numbers.
No primes will be found among them, that's for sure.
But we know that there must be at least one prime,
Since S is infinite, and that much is clear.

If we take the smallest number in S that is not in P,
And call it p, we can add it to our set, don't you see?
But this process can be repeated indefinitely.
Thus, our set P must also be infinite, you'll agree.

So there you have it, a proof that's quite solid.
There are infinitely many primes, that much we can see.
No need to be coy, no need to be contrite.
Infinity is a big number, just like P.
```

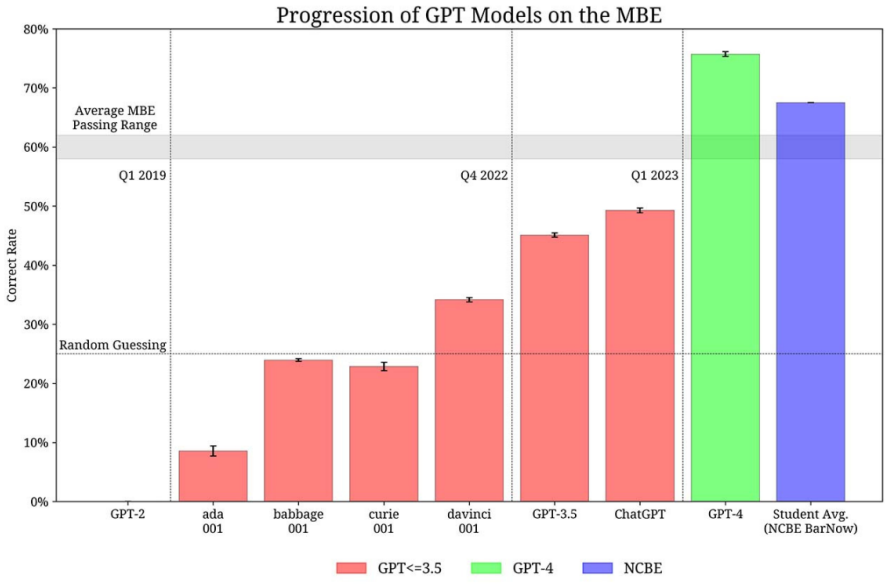
GPT-4:

```
Yes, I think I can,
Though it might take a clever plan.
I'll start by noting Euclid's proof,
Which shows that primes aren't just aloof.
Assume we have a finite list,
Of primes, and that none have been missed.
Multiply them all together,
And add one, just to be clever.
The result will be a number,
That has no prime factors, oh wonder!
But every number has a prime,
That divides it, it's just a matter of time.
So we've found a contradiction,
And our finite list needs eviction.
There must be infinitely many primes,
And that's the end of my rhyming lines.s
```

Prompt: Suppose $g(x) = f^{-1}(x), g(0) = 5, g(4) = 7, g(3) = 2, g(7) = 9, g(9) = 6$ what is $f(f(f(6)))$?

GPT-4: There are different ways to approach this problem, but one possible method is to use the fact that $g(x) = f^{-1}(x)$ implies that $f(g(x)) = x$ for any x in the domain of g. That is, applying f to both sides of the equation $g(x) = f^{-1}(x)$ gives $f(g(x)) = f(f^{-1}(x)) = x$. This means that we can undo the effect of f by applying g, and vice versa. Using this idea, we can try to find $f(6)$ by looking for the value of x such that $g(x) = 6$. From the given information, we see that $g(9) = 6$, so $f(6) = f(g(9)) = 9$. Similarly, we can find $f(9)$ by looking for the value of x such that $g(x) = 9$. We see that $g(7) = 9$, so $f(9) = f(g(7)) = 7$. Finally, we can find $f(7)$ by looking for the value of x such that $g(x) = 7$. We see that $g(4) = 7$, so $f(7) = f(g(4)) = 4$. Therefore, $f(f(f(6))) = f(f(9)) = f(7) = 4$.

GPT-4 crushing it on the U.S. Bar Exam



“Draw a unicorn in TikZ”



Figure 1.3: We queried GPT-4 three times, at roughly equal time intervals over the span of a month while the system was being refined, with the prompt “Draw a unicorn in TikZ”. We can see a clear evolution in the sophistication of GPT-4’s drawings.

and many, many, many more examples

- eg coding, vision, interactions, judgment...

How smart can RLHF LLMs get? Andrej Karpathy

(https://x.com/karpathy/status/1862656434361386l9)

- *People have too inflated sense of what it means to "ask an AI" about something. The AI are language models trained basically by imitation on data from human labelers.* ***Instead of the mysticism of "asking an AI", think of it more as "asking the average data labeler" on the internet.***
- *Few caveats apply because e.g. in many domains (e.g. code, math, creative writing)* ***the companies hire skilled data labelers. . .***

How smart can RLHF LLMs get? Andrej Karpathy

(https://x.com/karpathy/status/1862656434361386l9)

- *People have too inflated sense of what it means to "ask an AI" about something. The AI are language models trained basically by imitation on data from human labelers.* ***Instead of the mysticism of "asking an AI", think of it more as "asking the average data labeler" on the internet.***
- *Few caveats apply because e.g. in many domains (e.g. code, math, creative writing)* ***the companies hire skilled data labelers. . .***
- *But roughly speaking (and today),* ***you're not asking some magical AI. You're asking a human data labeler.*** *Whose average essence was lossily distilled into statistical token tumblers that are LLMs. . .*
- ***RLHF moves the performance to "discriminative human" grade, up from SFT which is at "generative human" grade.*** *But this is not so much "in principle" but more "in practice", because* ***discrimination is easier for an average person than generation*** *(e.g. label which of these 5 poems about X is best vs. write a poem about X). Separately* ***you also get a separate boost from the wisdom of crowds effect,*** *i.e. your LLM performance is not at human level, but at ensemble of human level.*

How smart can RLHF LLMs get? Andrej Karpathy

(https://x.com/karpathy/status/1862656434361386l9)

- *People have too inflated sense of what it means to "ask an AI" about something. The AI are language models trained basically by imitation on data from human labelers.* ***Instead of the mysticism of "asking an AI", think of it more as "asking the average data labeler" on the internet.***
- *Few caveats apply because e.g. in many domains (e.g. code, math, creative writing)* ***the companies hire skilled data labelers. . .***
- *But roughly speaking (and today),* ***you're not asking some magical AI. You're asking a human data labeler.*** *Whose average essence was lossily distilled into statistical token tumblers that are LLMs. . .*
- ***RLHF moves the performance to "discriminative human" grade, up from SFT which is at "generative human" grade.*** *But this is not so much "in principle" but more "in practice", because* ***discrimination is easier for an average person than generation*** *(e.g. label which of these 5 poems about X is best vs. write a poem about X). Separately* ***you also get a separate boost from the wisdom of crowds effect,*** *i.e. your LLM performance is not at human level, but at ensemble of human level.*
- ***So with RLHF in principle the best you can hope for is to reach a performance where a panel of e.g. the top 10 human experts on some topic, with enough time given, will pick your answer over any other.***

What should AI Value?

https://docs.google.com/presentation/d/1ekK_U3cAc8e76IzBf7Aq_nohI4b8F6ECp0nAit_kWd0/edit?usp=sharing